

A Controlled Empirical Study of Tabular Q-Learning and Deep Q-Network Variants for Cart-Pole Control

Chenglin Li

Participant 006

Session 1 Robotics Experiments

Abstract—Cart-pole balancing is a compact control task that exposes central reinforcement-learning concepts, including continuous observations, discrete actions, temporal-difference learning, exploration, and unstable value estimation. This paper presents a self-contained PyGame-compatible cart-pole environment and a controlled comparison of tabular Q-learning with deep Q-network (DQN) variants. Seven learned configurations are trained for an identical budget of 70,000 environment interactions under three random seeds. The study varies Q-learning discretization granularity and cumulatively introduces state normalization, Double DQN targets, and faster exploration decay to a common DQN implementation. Checkpoints are selected only from a separate validation set, while development and final evaluation sets contain disjoint initial-state seeds. On the final set, medium-resolution tabular Q-learning obtains the strongest result: 490.30 ± 9.94 mean episode steps and $91.67\% \pm 8.39\%$ success. The best DQN configuration by mean performance is the state-normalized variant, which reaches 472.10 ± 48.32 steps and $84.33\% \pm 27.14\%$ success. Double DQN does not provide a consistent gain across seeds in this setting. Moreover, every learned method degrades between its validation-selected best checkpoint and its checkpoint at 70,000 steps, with substantially larger degradation for the neural agents. These results show that representation scale, discretization density, repeated trials, and checkpoint policy can dominate apparently minor implementation choices on a small control problem.

I. Introduction

Reinforcement learning (RL) studies how an agent can improve sequential decisions through interaction. Unlike supervised learning, the desired action is not supplied at every state. Instead, the agent must connect present actions to delayed rewards while its own behavior changes the data that it will observe. Cart-pole balancing is a useful minimal example: a controller must apply a left or right force to a cart so that a hinged pole remains near the upright position without moving the cart beyond the track boundary. The task has been used in early adaptive-control research [1] and remains valuable because it has a continuous state, discrete actions, nonlinear dynamics, and a clear failure condition.

The apparent simplicity of cart-pole can conceal important experimental questions. A tabular controller requires a finite state representation, so a continuous observation must be discretized. A DQN avoids explicit discretization by approximating action values with a neural network [2], but introduces optimization, input scaling, replay-memory, and target-network choices. Double Q-learning

was developed to reduce the positive bias caused by maximizing noisy value estimates [3]; its deep variant separates action selection from target evaluation [4]. However, an algorithmic motivation does not guarantee an improvement on every task or implementation. Deep RL results can vary substantially across random seeds and experimental details [5].

This work therefore treats cart-pole as a controlled empirical study rather than a demonstration based on one successful run. The following research questions are considered:

- RQ1: How much do state normalization, Double DQN targets, and exploration-decay speed change DQN performance?
- RQ2: How does state discretization granularity affect tabular Q-learning under a fixed interaction budget?
- RQ3: Are the learned policies stable across training seeds and between the best and final checkpoints?

The study makes three practical contributions. First, it implements the cart-pole dynamics, training algorithms, evaluation protocol, and PyGame visualization without relying on a prebuilt Gym environment. Second, it compares four cumulative DQN configurations and three Q-learning discretizations using the same environment-step budget, three training seeds, and fixed validation and evaluation seeds. Third, it records both validation-selected best checkpoints and end-of-budget checkpoints, exposing training degradation that would be hidden by reporting only the strongest model. No new RL algorithm is claimed; the contribution is a reproducible, evidence-driven comparison of design choices.

II. Background

A. Markov Decision Process

The environment is modeled as a Markov decision process $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$. At time t , the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$, receives reward r_{t+1} , and observes s_{t+1} . The discounted return is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}, \quad (1)$$

where $\gamma \in [0, 1)$ controls the importance of future reward. The action-value function $Q^\pi(s, a)$ is the expected return after selecting a in s and subsequently following policy π .

B. Tabular Q-Learning

Q-learning is an off-policy temporal-difference control method [6]. For a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1})$, its tabular update is

$$\begin{aligned} \delta_t &= r_{t+1} + \gamma(1 - d_t) \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t), \\ Q(s_t, a_t) &\leftarrow Q(s_t, a_t) + \alpha \delta_t, \end{aligned} \quad (2)$$

where α is the learning rate and d_t indicates termination. The method is straightforward when every state-action pair can be represented in a table. For continuous cart-pole observations, each state dimension must instead be assigned to a finite bin. Coarse bins share experience across a large region but lose detail; fine bins preserve detail but create a much larger table whose entries may be visited too rarely.

C. Deep Q-Networks

DQN replaces the table with a neural approximation $Q(s, a; \theta)$. The network in this study maps four state values through two 128-unit ReLU layers to two linear action values. It is optimized from uniformly sampled replay transitions with the smooth L1 loss

$$\mathcal{L}(\theta) = \text{Huber}(Q(s_t, a_t; \theta), y_t), \quad (3)$$

where the standard DQN target is

$$y_t = r_{t+1} + \gamma(1 - d_t) \max_{a'} Q(s_{t+1}, a'; \theta^-). \quad (4)$$

Here, θ^- denotes a target network copied periodically from the online network. Experience replay reduces short-range correlation, and the target network slows movement of the bootstrap target, two mechanisms central to the original DQN design [2].

In Double DQN, the online network selects the next action while the target network evaluates it:

$$a^* = \arg \max_{a'} Q(s_{t+1}, a'; \theta), \quad (5)$$

$$y_t^{\text{DDQN}} = r_{t+1} + \gamma(1 - d_t) Q(s_{t+1}, a^*; \theta^-). \quad (6)$$

This separation can reduce maximization bias [4]. DQN still combines function approximation, bootstrapping, and off-policy updates, a combination associated with value instability [7]. Target networks can mitigate, but not eliminate, this interaction [8]. Consequently, repeated training runs and explicit checkpoint rules are necessary for credible comparison.

III. Method

A. Self-Contained Cart-Pole Environment

The environment state is

$$s_t = [x_t, \dot{x}_t, \theta_t, \dot{\theta}_t]^\top, \quad (7)$$

where x is cart position and θ is pole angle from vertical. The action space is $\mathcal{A} = \{0, 1\}$, corresponding to a force of -10 N or $+10$ N. The simulator uses gravity 9.8 m/s², cart

mass 1.0 kg, pole mass 0.1 kg, pole half-length 0.5 m, and a 0.02 s time step. Velocities are updated before positions using semi-implicit Euler integration. At reset, all four state values are sampled uniformly from $[-0.05, 0.05]$.

An episode terminates if $|x| > 2.4$ or $|\theta| > 12^\circ$. It succeeds if neither failure occurs for 500 steps. The reward function is

$$r_{t+1} = 1 + \begin{cases} -10, & |x_{t+1}| > 2.4 \text{ or } |\theta_{t+1}| > 12^\circ, \\ +10, & t+1 = 500 \text{ without failure,} \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

Thus, an ordinary transition yields $+1$, the failure transition yields -9 , and the successful terminal transition yields $+11$. The undiscounted episode return is $T - 10$ when failure occurs at step T , and 510 when the agent survives all 500 steps. Success rate and episode length are therefore more direct evaluation measures than return, which is nearly determined by survival time.

B. Tabular Controller

Position and pole angle are bounded by their termination thresholds. Velocity and angular velocity are clipped to $[-3, 3]$ and $[-3.5, 3.5]$, respectively, before discretization. Three bin vectors are evaluated:

$$\begin{aligned} \text{Coarse} &= [4, 4, 8, 8], \\ \text{Medium} &= [6, 6, 12, 12], \\ \text{Fine} &= [10, 12, 20, 20]. \end{aligned}$$

The table is updated with (2), using $\alpha = 0.2$ and $\gamma = 0.99$. The epsilon-greedy exploration rate decays exponentially from 1.0 toward 0.03 according to cumulative environment steps, with a decay scale of $15,000$ steps. Using environment steps rather than episodes avoids assigning a different exploration schedule merely because one policy survives longer.

C. DQN Controller

The online and target networks share a 4-128-128-2 multilayer perceptron architecture but hold separate parameters. Hidden layers use ReLU activation and orthogonal initialization; the output layer uses a smaller orthogonal gain. The online network is trained with Adam at 7.5×10^{-4} , discount $\gamma = 0.99$, batch size 64 , a replay capacity of $50,000$ transitions, and gradient-norm clipping at 10 . Learning begins after 500 transitions. The target network is copied every 500 environment steps.

For normalized variants, each input is divided element-wise by

$$c = [2.4, 3.0, 0.20944, 3.5]^\top. \quad (9)$$

This makes the typical scale of all four features comparable. Without this operation, the pole angle, whose failure boundary is about 0.20944 radians, has a substantially smaller numerical range than the translational features.

Table I defines the cumulative DQN study. The first three variants linearly decay epsilon from 1.0 to 0.05 over

TABLE I
Cumulative DQN configurations.

Variant	Norm.	Double	Decay steps
Standard DQN	No	No	50,000
Normalized DQN	Yes	No	50,000
Double DQN	Yes	Yes	50,000
Full DQN	Yes	Yes	12,000

50,000 steps. The full variant reaches the same endpoint after 12,000 steps. All other hyperparameters remain fixed.

D. Checkpoint and Evaluation Protocol

Each learned configuration is trained independently under seeds 42, 123, and 2026 for exactly 70,000 environment interactions. Validation is performed every 5,000 steps on 20 episodes beginning at seed 20,000. The checkpoint with the largest validation mean episode length is named best; the model after the complete budget is named final. Only the best checkpoint is used in the primary comparison, while the difference between best and final measures late-training degradation.

Two larger evaluation splits are kept separate. Development evaluation uses 100 initial-state seeds from 10,000 through 10,099. These seeds were observed during implementation and are not treated as unseen test data. Final evaluation uses seeds 30,000 through 30,099 and was executed only after the algorithm matrix and checkpoint rule were frozen. Metrics are averaged first within each 100-episode evaluation and then across the three training seeds. Reported deviations are sample standard deviations across training seeds, not standard errors over individual episodes. This convention exposes run-to-run sensitivity rather than hiding it behind 300 correlated episode measurements.

IV. Experiments

A. Experimental Setup

Training ran on Windows with Python 3.10.19, NumPy 1.26.4, PyTorch 2.6.0+cu124, and an NVIDIA GeForce RTX 4060 Laptop GPU. The complete matrix contains $7 \times 3 = 21$ learned runs and 1,470,000 environment interactions. Each configuration-seed pair was trained exactly once, and the complete 7×3 matrix was not independently repeated. Consequently, the three observations reported for each learned method are single training runs under seeds 42, 123, and 2026, rather than repeated runs under the same seed. A random policy is evaluated under the same episode rules as an untrained reference. The source tree records a configuration hash, fixed seeds, checkpoint metadata, per-episode diagnostics, validation curves, and machine-readable evaluation files.

The primary metrics are mean episode length and the fraction of episodes reaching 500 steps. Training return, epsilon, and temporal-difference loss are treated only as

diagnostics. This distinction is important because a small loss does not imply that the greedy policy balances the pole, and return is almost a linear transformation of episode length in this reward design.

B. Final Performance

Table II and Fig. 1 report final-set performance for validation-selected best checkpoints. Medium-resolution Q-learning is strongest on both metrics, obtaining 490.30 ± 9.94 steps and $91.67\% \pm 8.39\%$ success. It exceeds Full DQN by 20.31 mean steps and 12.00 percentage points of success. Coarse Q-learning is also stable, whereas fine Q-learning has a much larger standard deviation.

C. DQN Ablation: Representation Dominates

Standard DQN improves substantially over the random policy in episode length but never reaches the 500-step success criterion. Adding only state normalization raises mean length by 252.93 steps and success by 84.33 percentage points. This is the largest incremental DQN improvement, answering RQ1: input scale is critical for this compact MLP and state definition.

Adding Double DQN after normalization does not improve the aggregate result. Seeds 42 and 2026 both achieve 100% final-test success with their best checkpoints, but seed 123 achieves only 18%. The resulting standard deviation of 161.97 steps is more than three times that of normalized DQN. Faster exploration decay in Full DQN recovers mean performance to 469.99 steps and reduces the episode-length standard deviation to 30.58. Nevertheless, normalized DQN remains slightly stronger in both mean length and success. Thus, the experiment supports neither a universal advantage for Double DQN nor a claim that the cumulative full configuration is best. The intended reduction of maximization bias addresses one failure mechanism, while representation and optimization stability remain task dependent.

D. Q-Learning Discretization

Medium bins provide the best balance between resolution and visitation frequency, answering RQ2. Coarse discretization is highly repeatable but loses some control precision: its success ranges from 74% to 88% across seeds. Medium discretization ranges from 82% to 97%. Fine discretization performs well for seeds 42 and 123, at 87% and 98%, but fails entirely for seed 2026, whose best checkpoint reaches only 174.31 mean steps. The fine table therefore produces 381.90 ± 179.87 steps. Increasing representational capacity without increasing the interaction budget creates sparse state-action coverage and can make a seemingly more precise controller less reliable.

E. Checkpoint Degradation

Figure 2 and Table III answer RQ3. Every method is weaker at 70,000 steps than at its validation-selected best checkpoint. The three strongest DQN variants lose approximately 345–352 mean steps. Tabular methods also

TABLE II

Final evaluation of validation-selected best checkpoints. Values are mean $\pm$ sample standard deviation across three training seeds; each seed is evaluated for 100 episodes.

Family	Method	Mean episode length	Success rate	Mean train time (s)
Reference	Random policy	20.53 $\pm$ 1.21	0.00% $\pm$ 0.00%	–
DQN	Standard DQN	219.17 $\pm$ 64.01	0.00% $\pm$ 0.00%	253.92
DQN	Normalized DQN	472.10 $\pm$ 48.32	84.33% $\pm$ 27.14%	378.98
DQN	Double DQN	406.49 $\pm$ 161.97	72.67% $\pm$ 47.34%	231.63
DQN	Full DQN	469.99 $\pm$ 30.58	79.67% $\pm$ 26.27%	219.75
Tabular	Q-Learning Coarse	482.78 $\pm$ 9.10	82.67% $\pm$ 7.57%	6.47
Tabular	Q-Learning Medium	490.30 $\pm$ 9.94	91.67% $\pm$ 8.39%	6.63
Tabular	Q-Learning Fine	381.90 $\pm$ 179.87	61.67% $\pm$ 53.69%	5.79

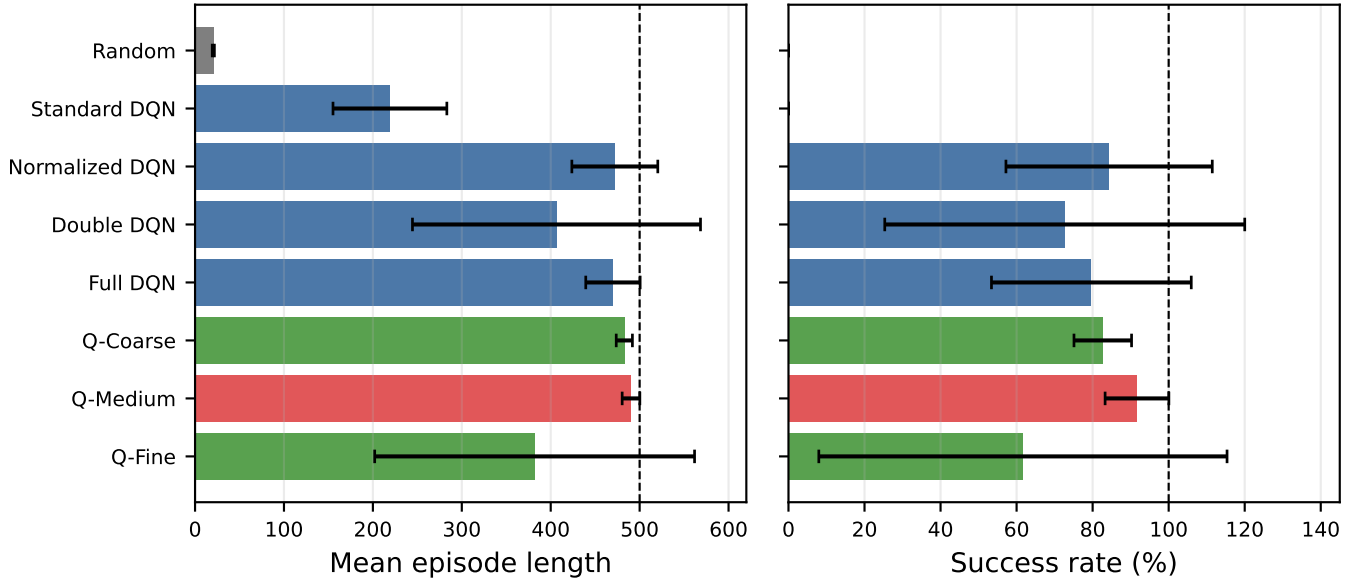


Fig. 1. Final-set performance of best checkpoints. Error bars show one standard deviation across three training seeds. Dashed lines mark the 500-step and 100% limits.

TABLE III
Best-to-final checkpoint degradation.

Method	Best	Final	Gap
Standard DQN	219.17	90.01	129.16
Normalized DQN	472.10	124.33	347.77
Double DQN	406.49	54.56	351.92
Full DQN	469.99	124.95	345.04
Q-Coarse	482.78	327.44	155.34
Q-Medium	490.30	341.33	148.97
Q-Fine	381.90	202.55	179.35

degrade, but their mean gaps are 149–179 steps. The largest DQN collapse is Double DQN, whose final checkpoint averages only 54.56 steps despite a best-checkpoint mean of 406.49.

The best checkpoint occurs anywhere from 5,000 to 70,000 steps, and its location varies strongly even within one method. For example, Full DQN is selected at 65,000

steps for seeds 42 and 123 but at 5,000 steps for seed 2026. The validation curves in Fig. 3 are correspondingly non-monotonic. A fixed end-of-training policy is therefore not a defensible substitute for validation-based model selection in this experiment.

The behavior is consistent with known reproducibility concerns in deep RL [5] and with instability mechanisms that can arise when off-policy learning, function approximation, and bootstrapping interact [7]. The tabular agents do not use function approximation, yet their nonstationary epsilon-greedy data distribution and constant learning rate can still overwrite useful action values. The result does not identify one exclusive cause, but demonstrates that a checkpoint policy materially changes the reported conclusion.

F. Development-to-Test Agreement and Runtime

The ranking is not an artifact of one evaluation seed range. Across all learned methods, final-test mean episode

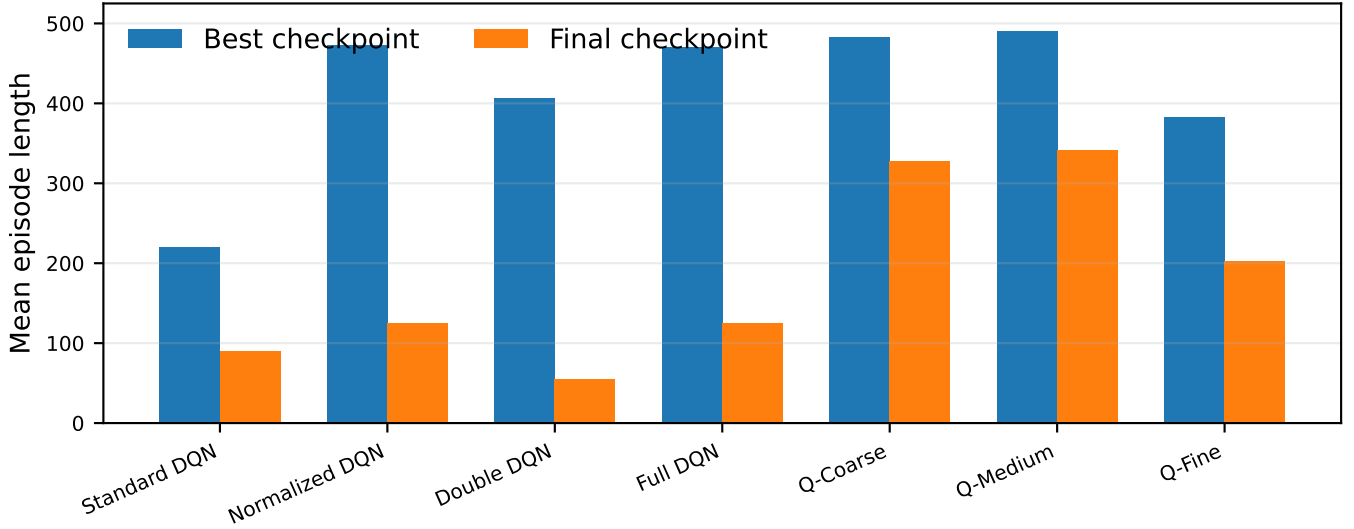


Fig. 2. Mean final-set episode length for validation-selected best and 70,000-step final checkpoints. Continued training degrades every method.

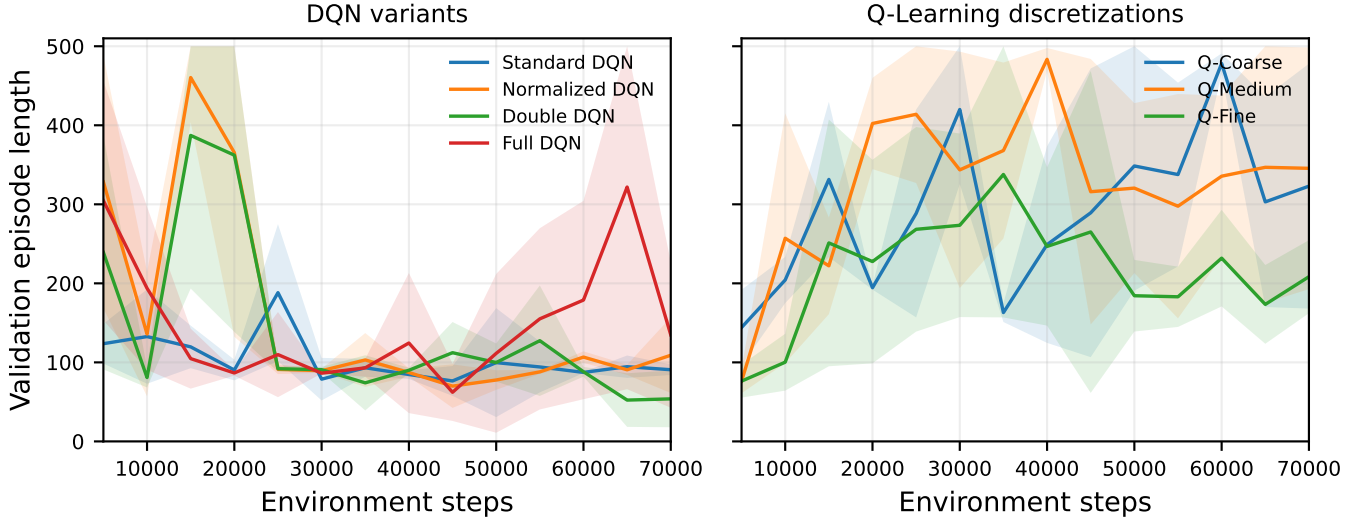


Fig. 3. Validation episode length during training. Lines show the mean over three training seeds and shaded regions show one standard deviation, clipped to the valid 0–500 range. Both families exhibit non-monotonic learning.

length differs from development evaluation by only -9.49 to $+6.74$ steps. Q-Medium changes from 495.33 steps and 94.00% success on development states to 490.30 and 91.67% on final states. Full DQN changes from 475.41 and 81.33% to 469.99 and 79.67%. These small shifts support the use of the final values as a generalization check, while the configuration remains frozen after their release.

The 21 training records contain 55.16 minutes of accumulated wall-clock time. Individual DQN runs average 220–379 seconds, compared with approximately six seconds for a tabular run. These measurements describe this implementation and hardware rather than theoretical complexity: DQN performs batched backpropagation at every

eligible environment step, whereas Q-learning updates one table entry.

G. Threats to Validity

Only three training seeds are available, so the estimated between-run standard deviations are themselves uncertain. Each checkpoint is selected from 20 validation episodes, which can favor a noisy local peak. The 500-step horizon causes ceiling effects among strong policies; success rate is useful but also saturates. Conclusions apply to this self-contained cart-pole implementation and fixed budget, not automatically to image-based control or environments with larger action spaces. No confidence interval or hy-

pothesis test can compensate for the small number of independently trained agents, so the paper reports effect sizes and individual-seed behavior rather than claiming statistical significance.

The formal matrix was executed only once: there is one training trajectory for every method–seed pair and no second execution of the full protocol. The experiment therefore measures sensitivity across three random seeds but does not directly measure rerun reproducibility under an identical seed and software–hardware state. Additional independent repetitions are needed before making stronger claims about run-to-run stability.

Finally, the cumulative DQN sequence changes one intended factor at each stage, but the label “Double DQN” should not be interpreted as isolating maximization bias itself; only task performance is observed. Similarly, the fast exploration schedule is one schedule choice, not a complete exploration study. These boundaries motivate future work with more training seeds, learning-rate ablations, larger validation sets, adaptive stopping, and a new sealed test split if any hyperparameter is changed.

V. Conclusion

This paper compared tabular Q-learning and DQN variants for a self-contained cart-pole control task under a common 70,000-step budget. Medium-resolution Q-learning delivered the best and most consistent final result. Among neural agents, state normalization was the decisive modification, while Double DQN did not yield a consistent cross-seed improvement. Fine tabular discretization also failed to generalize across training seeds, demonstrating that additional representational capacity can be counterproductive when experience is limited.

The strongest lesson is methodological. Every configuration degraded between its validation-selected best checkpoint and its final checkpoint, and several DQN policies collapsed after previously solving the task. Reporting one seed or only the end-of-budget model would therefore produce a misleading account. Controlled interaction budgets, disjoint evaluation seeds, repeated runs, and explicit checkpoint selection are not secondary bookkeeping choices; they determine whether a small RL experiment supports a credible conclusion.

References

- [1] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] H. van Hasselt, “Double q-learning,” in *Advances in Neural Information Processing Systems*, vol. 23, 2010, pp. 2613–2621.
- [4] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016, pp. 2094–2100.
- [5] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 3207–3214.
- [6] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [7] H. van Hasselt, Y. Doron, F. Strub, M. Hessel, N. Sonnerat, and J. Modayil, “Deep reinforcement learning and the deadly triad,” *arXiv preprint arXiv:1812.02648*, 2018.
- [8] S. Zhang, H. Yao, and S. Whiteson, “Breaking the deadly triad with a target network,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 139, 2021, pp. 12 621–12 631.